

GCN Attack Detection Model Evolution

v1.0.0, v2.0.0, v2.1.0, and v3.0.0

Comprehensive version history, inputs, architecture, improvements, limitations, and production-readiness analysis.

Prepared as a single reference document for copy-paste into reports, README files, documentation, or thesis notes.

GCN Attack Detection Model Evolution: v1.0.0, v2.0.0, v2.1.0, and v3.0.0

Document Purpose

This document gives a full, structured analysis of the Graph Convolutional Network (GCN) attack detection models used in the Wi-Fi 7 MLO security pipeline. It explains:

- what each GCN version is
- what inputs are fed to the model
- how each version was trained and integrated
- what improved from one version to the next
- what limitations remained
- how the versions compare
- which version is best for production

This is written as a single reference document so it can be copied directly into a report, README, project documentation, or thesis notes.

1. Introduction

The GCN-based attack detector was designed to identify **Wi-Fi 7 MLO backoff manipulation attacks** in a digital twin pipeline. The core idea is to transform time-series telemetry into a graph structure, then use a Graph Convolutional Network to classify traffic as either:

- ****Class 0 = Normal****
- ****Class 1 = Attack****

Over time, multiple GCN versions were developed because early models worked well in isolated training and test environments, but did not always generalize correctly to the real pipeline. This led to several important lessons about:

- data distribution mismatch
- class imbalance
- model generalization
- feature processing consistency
- multi-AP support
- variable segment-length support
- proper production evaluation

The major versions discussed here are:

- **v1.0.0** – original baseline production model
- **v2.0.0** – first retraining/integration attempt on pipeline-aligned effort
- **v2.1.0** – retrained fix attempt after v2.0.0 false-positive issues
- **v3.0.0** – improved model with multi-AP and variable segment-length support, fully evaluated and production-ready

2. What the GCN Model Receives as Input

2.1 Raw Data Source

The GCN does not directly read raw ns-3 events one by one. Instead, the pipeline first collects telemetry and organizes it into windows.

The telemetry comes from:

- **ns-3 Wi-Fi 7 MLO simulations**
- potentially future **live Wi-Fi sensor streams**

The telemetry includes per-window measurements such as throughput, delay, jitter, packet loss, MAC counters, PHY counters, backoff behavior, and channel occupancy.

2.2 Base Features Fed to the GCN

Across the versions, the main base feature set used by the GCN consisted of **13 base features**:

1. ``net_throughput_mbps``
2. ``net_avg_delay_ms``
3. ``net_avg_jitter_ms``
4. ``net_packet_loss_ratio``
5. ``net_active_flows``
6. ``mac_tx_delta`` or derived from ``mac_total_tx``
7. ``mac_rx_delta`` or derived from ``mac_total_rx``
8. ``mac_ack_delta`` or derived from ``mac_total_ack``
9. ``mac_retrans_delta`` or derived from ``mac_total_retrans``
10. ``mac_drop_delta`` or derived from ``mac_drop_count``
11. ``phy_drop_delta`` or derived from ``phy_drop_count``
12. ``avg_backoff_slots``
13. ``channel_busy_ratio``

Important note:

- in raw datasets, the MAC counters often appeared as cumulative counters such as `mac\_total\_tx`, `mac\_total\_rx`, etc.
- before being fed into the GCN, these cumulative values were converted into **per-window delta features**
- this delta conversion was a very important preprocessing step

2.3 Derived Features

Most versions also used **3 derived features**, bringing the total input dimension to **16 features**:

1. $\text{retrans_rate} = \text{mac_retrans_delta} / \text{mac_tx_delta}$
2. $\text{drop_rate} = (\text{mac_drop_delta} + \text{phy_drop_delta}) / \text{mac_tx_delta}$
3. $\text{ack_ratio} = \text{mac_ack_delta} / \text{mac_tx_delta}$

In some planning documents, `throughput_per_flow` was also discussed as a possible derived feature, but the main documented training versions consistently referenced the above derived-rate style features.

Important observation from the original training data:

- many MAC counters were always zero in the old datasets
- therefore, several derived features defaulted to zero
- this reduced the practical importance of some MAC-derived inputs in early versions

2.4 What Must Never Be Used as Input

The **bias field** indicates whether the scenario is normal or attack and how strong the attack is.

Examples:

- `bias = 0` → normal
- `bias = +5000` → positive attack
- `bias = -500` → negative attack

This field was used only for **labels** during training.

Critical rule:

The `bias` field must **never** be used as an input feature, because that would create **data leakage** and the model would effectively cheat.

3. How Data Is Structured Before Entering the GCN

3.1 Window Structure

Each simulation file contains many windows. A typical window looks like:

```
{
  "window": 100,
  "bias": 0,
  "net_throughput_mbps": 382.676,
  "net_avg_delay_ms": 218.242,
  "net_avg_jitter_ms": 3.1598,
  "net_packet_loss_ratio": 0.844865,
  "net_active_flows": 9,
  "mac_total_tx": 0,
  "mac_total_rx": 0,
  "mac_total_ack": 0,
  "mac_total_retrans": 0,
  "mac_drop_count": 0,
  "phy_drop_count": 0,
  "avg_backoff_slots": 13.5008,
  "channel_busy_ratio": 0.900348
}
```

Each file contains a sequence of such windows over time.

3.2 Segmenting the Time Series

The GCN does not classify a single window alone. Instead, windows are grouped into **segments**.

Typical segment sizes used across versions:

- ****256 windows**** – the classic default
- ****128 windows**** – later supported in v3.0.0
- ****64 windows**** – later supported in v3.0.0
- 32-window support was also discussed in training plans for later versions

A 256-window segment means:

- 256 consecutive telemetry windows are grouped
- each window becomes a node in the graph
- one graph is then classified as normal or attack

So:

- ****one segment = one graph****
- ****one window = one node****

3.3 Graph Construction

Each segment is converted into a temporal chain graph:

- **nodes** = windows
- **edges** = sequential time connections

For a 256-window segment:

- nodes = 256
- edges connect node i to node $i+1$
- usually bidirectional edges are added, so the graph can look both forward and backward in local time context

For example:

- node 0 $\leftrightarrow$ node 1
- node 1 $\leftrightarrow$ node 2
- node 2 $\leftrightarrow$ node 3
- ...
- node 254 $\leftrightarrow$ node 255

This gives the GCN temporal structure without requiring an RNN.

4. Common GCN Architecture Across Versions

Although versions changed in support and training behavior, the core architecture remained broadly similar.

4.1 Baseline Architecture

Typical architecture:

- **Input channels**: 16
- **Hidden channels**: 64
- **Number of GCN layers**: 2
- **Dropout**: 0.3
- **Pooling**: mean or global graph pooling
- **Output**: 2-class softmax (Normal vs Attack)

Typical components:

1. Input node features

2. GCNConv Layer 1
3. BatchNorm + ReLU
4. GCNConv Layer 2
5. Residual/skip connection in later improved forms
6. Global graph pooling
7. MLP classifier head
8. Softmax probabilities

4.2 Output

For each graph/segment, the model outputs:

- ``prediction = 0` or `1``
- ``confidence``
- ``probabilities = [p_normal, p_attack]``

Decision rule:

- if ``p_attack > 0.5``, classify as **Attack**
- otherwise classify as **Normal**

5. Version-by-Version Analysis

5.1 GCN v1.0.0

5.1.1 Overview

v1.0.0 was the **original baseline GCN model** used before the later integration failures were deeply analyzed. It came from the original `wifi7_gcn_attack_detection` training repository and served as the first working graph-based attack detector.

This model showed very strong offline performance, but later analysis suggested that its training distribution did not match the pipeline's real production-like data.

5.1.2 Training Data Used in v1.0.0

The original training dataset had:

- **12 normal files**
- **192 attack files**
- total **204 files**

Distribution:

- Normal = **5.9%**
- Attack = **94.1%**

Attack distribution:

- Positive bias = 96 files
- Negative bias = 96 files

This means the original dataset was **highly imbalanced toward attack**.

5.1.3 Bias Values Used in v1.0.0

The original dataset used **8 logarithmically spaced bias levels**:

- 50
- 100
- 250
- 500
- 1000
- 2500
- 5000
- 10000

Each bias level had:

- 12 positive scenarios
- 12 negative scenarios
- 24 attack scenarios total

This was actually one of the strengths of the original dataset: it covered both subtle and strong attacks.

5.1.4 Scenario Diversity in v1.0.0

The original training data used **4 network scenarios** representing increasing network density:

1. ****Light network****

- ~6 active flows
- ~412 Mbps throughput

2. ****Moderate network****

- ~8.7 active flows
- ~325 Mbps throughput

3. ****Dense network****

- ~10.2 active flows
- ~303 Mbps throughput

4. ****Very dense network****

- ~11.9 active flows
- ~261 Mbps throughput

This gave good diversity in network density, but later the problem was not lack of density diversity. The problem was that the pipeline's actual normal traffic still did not match the old "normal" distribution closely enough.

5.1.5 Strengths of v1.0.0

- First working baseline GCN attack detector
- Covered subtle and strong bias levels
- Used multiple network-density scenarios
- Used multiple sessions and seeds to add randomness
- Achieved very high reported offline performance

5.1.6 Limitations of v1.0.0

- Heavily attack-dominated training distribution
- Very little normal data
- Likely learned a biased representation of normal traffic
- Poor generalization to pipeline-produced normal traffic
- Throughput and other feature distributions did not match later production conditions
- High offline performance did not guarantee production readiness

5.1.7 Summary of v1.0.0

v1.0.0 was an important baseline model that proved GCN-based attack detection could work. However, it relied on a highly imbalanced training dataset and learned a version of “normal traffic” that did not align well with later pipeline data. Its main contribution was establishing the graph-based detection approach and the important role of bias-level coverage.

5.2 GCN v2.0.0

5.2.1 Overview

v2.0.0 was the first major retraining/integration attempt after the original baseline. It was trained using the same general GCN architecture and achieved extremely strong test-set metrics during training, but failed badly during deployment.

It became one of the most important learning points in the project because it showed that **excellent offline test accuracy can still hide serious production failure**.

5.2.2 Training Configuration

Typical configuration documented for v2.0.0:

- max_epochs: 50
- batch_size: 32
- learning_rate: 0.001
- patience: 10
- random_seed: 42
- device: cuda
- segment_length: 256
- stride: 256
- in_channels: 16
- hidden_channels: 64
- num_layers: 2
- dropout: 0.3
- pooling: mean
- optimizer: Adam
- weight_decay: 0.0001

5.2.3 Dataset Balance at File Level vs Segment Level

File-level split:

- total files: 204
- attack: 192
- normal: 12

Train/Val/Test split at file level remained highly imbalanced.

But after segmentation into 256-window chunks, the segment counts looked much more balanced:

- Train: 834 segments
- 432 normal
- 402 attack

This created the impression that training balance was acceptable, but the deeper issue turned out not to be only class count. It was also about **distribution mismatch**.

5.2.4 Reported Training Results

v2.0.0 achieved extremely strong offline metrics:

- Accuracy: 1.0000
- Precision: 1.0000
- Recall: 1.0000
- F1 Score: 1.0000
- ROC-AUC: 1.0000

Confusion matrix:

- 108 true negatives
- 87 true positives
- 0 false positives
- 0 false negatives

At first glance, this looked perfect.

5.2.5 Why v2.0.0 Was a Red Flag

Although these results looked excellent, later investigation showed the model was not truly production-ready.

Standalone validation already gave a warning:

- normal file → predicted attack with 99.74% confidence
- attack file → predicted attack correctly

So even before full pipeline integration, the model was already biased toward attack in some cases.

5.2.6 Pipeline Integration Failure of v2.0.0

When v2.0.0 was deployed into the digital twin pipeline:

- normal traffic was classified as attack in ****100% of normal segments****
- attack traffic was also classified as attack

So effectively:

- normal = attack
- attack = attack

This means v2.0.0 had **100% false positive rate on normal pipeline traffic**.

5.2.7 Pipeline Issues Investigated

During debugging, two real pipeline issues were found:

A. Windowizer field name mismatch

Pipeline used:

- `mac\_total\_tx`, `mac\_total\_rx`, etc.

Training expected delta-style fields:

- `mac\_tx\_delta`, `mac\_rx\_delta`, etc.

This was fixed.

B. Feature processor mismatch

The detector feature processor was also using old field names.

This was fixed too.

However, even after both fixes, v2.0.0 still predicted normal traffic as attack.

So the final conclusion was:

> the model itself was not robust enough for production conditions

5.2.8 Real Root Cause Identified Later

The biggest issue was not only the field-name mismatch. The deeper problem was:

Training “normal” data was unrealistic

Training normal traffic had feature distributions like:

- packet loss $\approx$ 83.8%
- delay $\approx$ 124 ms
- backoff slots $\approx$ 18.0

But pipeline validation normal traffic had:

- packet loss $\approx$ 2.5%
- delay $\approx$ 4.3 ms
- backoff slots $\approx$ 9.7

So the model had learned:

- ****normal = degraded/high-loss traffic****
- ****attack = low-backoff traffic****

When it later saw healthy realistic normal traffic, it said:

> “this doesn’t look like the normal I know; it must be attack”

That is why v2.0.0 failed.

5.2.9 Strengths of v2.0.0

- Strong offline performance
- Correct graph-processing pipeline structure
- Helped uncover important integration and data-distribution issues

- Revealed the danger of relying only on held-out test metrics

5.2.10 Limitations of v2.0.0

- Failed on real pipeline normal traffic
- Extremely high false positive rate in deployment
- Training normal data did not match realistic normal traffic
- Offline metrics were misleading
- Not production-ready

5.2.11 Summary of v2.0.0

v2.0.0 was a major learning step. It looked perfect in offline testing, but failed badly in the real pipeline because the training data distribution did not represent realistic normal traffic. It taught the project team that production realism matters more than perfect offline metrics.

5.3 GCN v2.1.0

5.3.1 Overview

v2.1.0 was a retrained follow-up model intended to fix the catastrophic false-positive behavior of v2.0.0. It performed much better in standalone testing and initially appeared to solve the issue.

However, later deeper investigation showed that although v2.1.0 was better than v2.0.0 in isolated testing, it still did not fully solve the production distribution problem.

5.3.2 Training Results of v2.1.0

Reported test metrics:

- Accuracy: 0.9949
- Precision: 0.9886
- Recall: 1.0000
- F1 Score: 0.9943
- ROC-AUC: 1.0000

Confusion matrix:

- 107 true negatives
- 87 true positives
- 1 false positive
- 0 false negatives

This looked more realistic than v2.0.0's suspicious 100%.

5.3.3 Standalone Validation of v2.1.0

Standalone test on 6 files:

- 3/3 normal files correctly predicted as normal
- 3/3 attack files correctly predicted as attack

This seemed to confirm that v2.1.0 had solved the v2.0.0 attack-all-the-time problem.

5.3.4 Initial Deployment Interpretation

At one stage, deployment notes claimed:

- v2.1.0 fixed the 100% false positive issue
- normal traffic accuracy improved from 0% to 100%
- false positive rate dropped to 0.93%
- attack detection stayed at 100%

This was true **within the scope of the evaluated standalone and test-set conditions**.

5.3.5 What Was Later Discovered

After deeper end-to-end investigation, it became clear that:

- v2.1.0 still failed in the real pipeline on realistic normal traffic
- standalone success did not fully transfer to production
- the real root cause remained the mismatch between old training normal traffic and realistic pipeline normal traffic

So v2.1.0 was an improvement over v2.0.0, but not the final solution.

5.3.6 Strengths of v2.1.0

- More realistic metrics than v2.0.0
- Better standalone validation performance
- Lower false positive rate on held-out offline tests
- Demonstrated that retraining could improve behavior
- Confirmed that the architecture itself was not fundamentally hopeless

5.3.7 Limitations of v2.1.0

- Still inherited the broader distribution mismatch problem
- Standalone success did not guarantee pipeline success
- Did not fully solve production false positives
- Still insufficient as the final production model

5.3.8 Summary of v2.1.0

v2.1.0 was a useful improvement over v2.0.0 and reduced obvious false-positive bias in standalone evaluation, but it did not fully solve the deeper problem of unrealistic training data. It was an intermediate recovery version, not the final production answer.

5.4 GCN v3.0.0

5.4.1 Overview

v3.0.0 is the most advanced and most thoroughly evaluated version described in the available documentation. It was designed to improve beyond earlier versions by supporting:

- multi-AP environments
- multiple segment lengths
- better generalization across seeds
- broader deployment realism

It is the version ultimately recommended for production.

5.4.2 Major Improvements Introduced in v3.0.0

Compared with earlier versions, v3.0.0 added:

1. **Multi-AP support**

- tested at 1 AP, 2 AP, and 4 AP
- 3 AP was trained but not explicitly tested in the full matrix
- later nap5/6 planned for future versions

2. **Variable segment-length support**

- works at 256 windows
- works at 128 windows
- works at 64 windows

3. **Segment-length conditioning feature**

- a 17th conditioning feature based on segment length was introduced
- typically described as $\log_2(L) / 8.0$
- this allowed one model to adapt across multiple temporal context sizes

4. **Per-AP normalization**

- throughput and some MAC/PHY deltas were normalized relative to AP count or station count
- this allowed the model to generalize across topology sizes

5.4.3 Evaluation Methodology for v3.0.0

v3.0.0 was not only trained; it was also evaluated using a systematic **5-tier evaluation matrix**.

The five evaluation tiers were:

1. Core Accuracy
2. Multi-AP Scaling
3. Segment Length Sensitivity
4. Bias Sensitivity
5. Seed Generalisation

This is one of the strongest reasons v3.0.0 is considered production-ready: it was comprehensively tested under many controlled conditions.

5.4.4 Evaluation Results of v3.0.0

Full matrix total:

- ****54/54 PASS****
- 0 failures

Tier 1 – Core Accuracy

- perfect binary detection at 1 AP / 256-window
- normal attack_rate = 0.000
- attack attack_rate = 1.000

Tier 2 – Multi-AP

- perfect detection at:
 - 2 AP / 4 STA
 - 4 AP / 8 STA

Tier 3 – Segment Length

- perfect detection at:
 - 128-window
 - 64-window

Tier 4 – Bias Sensitivity

- attacks at bias = 1000 detected reliably
- no degradation from 10000 down to 1000

Tier 5 – Seed Generalisation

- stable results across five seed groups A–E

5.4.5 Comparison to v2.0.0 in Final Evaluation

From the final evaluation guide and report:

v2.0.0

- worked perfectly at its supported configuration:
 - 1 AP
 - 256-window
- showed higher attack confidence in that narrow configuration
- not designed for multi-AP or variable segment lengths

v3.0.0

- matched v2.0.0's binary detection in the base configuration
- generalized to:

- 2 AP
- 4 AP
- 64/128/256 windows
 - remained robust across multiple seed groups
 - therefore became the preferred production version

5.4.6 Strengths of v3.0.0

- Fully evaluated with systematic 54-experiment matrix
- Supports multi-AP topologies
- Supports variable segment lengths
- Robust across seeds
- Detects low-bias attacks reliably
- Production-ready
- Most general and flexible model in the documented history

5.4.7 Limitations of v3.0.0

- Some negative-scenario attack confidences varied by seed
- Not yet evaluated for AP counts beyond trained distribution such as 5 or 6 AP
- Requires careful per-AP normalization and conditioning features to remain stable
- As always, future deployment should still monitor drift if real environments evolve

5.4.8 Summary of v3.0.0

v3.0.0 is the strongest GCN version described in the project history. It improved beyond earlier versions by supporting multi-AP topologies, multiple segment lengths, and stronger generalization across random conditions. It passed all 54 experiments in the final evaluation matrix and is the recommended production model.

6. What Improved from Version to Version

6.1 v1.0.0 → v2.0.0

Improvements attempted

- retraining and integration refinement
- stronger formal training pipeline
- explicit segment-level dataset handling
- model registry integration

What went wrong

- production generalization failed badly
- field mismatch issues exposed
- data distribution mismatch remained hidden

Net effect

v2.0.0 did not become a better production model than v1.0.0 despite excellent offline metrics. Its real contribution was exposing the mismatch problem.

6.2 v2.0.0 → v2.1.0

Improvements

- retrained from scratch
- much better standalone normal/attack separation
- more realistic metrics
- reduced obvious false-positive behavior in isolated evaluation

Remaining limitation

- still did not fully solve the realistic production-data distribution mismatch

Net effect

v2.1.0 was an improvement, but still transitional.

6.3 v2.x → v3.0.0

Major improvements

- multi-AP support
- variable segment-length support
- segment-length conditioning
- per-AP feature normalization
- broader evaluation coverage
- comprehensive 5-tier evaluation matrix
- stronger confidence in production readiness

Net effect

v3.0.0 is clearly the most mature and most deployable version.

7. Limitations of Each Version

v1.0.0

- original normal data distribution did not match later pipeline reality
- heavily attack-imbalanced dataset
- not demonstrated as strong in production realism

v2.0.0

- catastrophic false positives in pipeline
- misleadingly perfect offline metrics
- realistic normal traffic not represented correctly in training

v2.1.0

- better than v2.0.0, but still not the final distributional fix
- standalone improvement did not fully translate to real pipeline

v3.0.0

- strongest version, but still bounded by tested AP-count ranges
- future larger topologies and later deployment conditions still need monitoring

8. Side-by-Side Comparison Table

Dimension	v1.0.0	v2.0.0	v2.1.0	v3.0.0
Main role	Original baseline	First major retraining/integration attempt	Retrained fix attempt after v2.0.0	Production-ready improved version
Core architecture	2-layer GCN	2-layer GCN	2-layer GCN	2-layer GCN with conditioning/generalization improvements
Input features	13 base + derived features	13 base + 3 derived = 16	13 base + 3 derived = 16	16 + segment-length conditioning feature
Segment length	256	256	256	64, 128, 256

Dimension	v1.0.0	v2.0.0	v2.1.0	v3.0.0
Multi-AP support	Not documented as supported	No	No	Yes
Offline test metrics	Very strong	Perfect/suspicious	Very strong and more realistic	Perfect pass matrix
Production reliability	Limited	Poor	Improved but incomplete	Strong
Main issue	Imbalanced / old distribution	100% false positives in pipeline	Still affected by distribution mismatch	Best evaluated, broadest support
Bias coverage	Strong	Strong	Strong	Strong
Seed generalisation	Limited documentation	Limited	Limited	Explicitly tested across A–E
Final status	Baseline only	Not production-ready	Transitional improvement	Recommended production model

9. Inputs Fed to the GCN: Final Consolidated Explanation

The GCN ultimately receives, for each segment:

1. a set of **window-level features**
2. grouped into a segment of length **L**
3. converted into a graph
4. then classified at graph level

Typical window features

- throughput
- delay
- jitter
- packet loss
- active flows
- MAC TX/RX/ACK/retrans/drop deltas
- PHY drops
- average backoff slots
- channel busy ratio
- derived rates

Graph structure

- one node per window
- sequential temporal edges
- one graph per segment

Model output

- class label
- confidence
- attack probability

10. Overall Evolution Story

The GCN model history teaches a very important engineering lesson.

At first, the team focused mainly on:

- model accuracy
- test-set metrics
- graph architecture

But deployment revealed that these alone are not enough.

The real success factors turned out to be:

- realistic training data
- correct feature engineering
- production-aligned distributions
- topology generalization
- segment-length generalization
- systematic end-to-end evaluation

In that sense:

- ****v1.0.0**** proved the concept
- ****v2.0.0**** exposed the hidden problems
- ****v2.1.0**** partially improved them
- ****v3.0.0**** solved the most important limitations and became production-ready

11. Final Summary of Each GCN Version

v1.0.0 Summary

v1.0.0 was the original baseline GCN attack detector trained on the first large Wi-Fi 7 attack dataset. It established the core graph-based approach and used a strong set of attack bias levels and network-density scenarios. However, it relied on a highly imbalanced training set and learned a version of normal traffic that did not align well with later pipeline data.

v2.0.0 Summary

v2.0.0 was a major retraining and integration effort that achieved apparently perfect offline metrics, but failed badly in production by classifying all normal traffic as attack. It revealed the critical importance of realistic training data and showed that perfect held-out metrics do not guarantee successful deployment.

v2.1.0 Summary

v2.1.0 was a retrained fix attempt that improved standalone validation and reduced the obvious false-positive bias seen in v2.0.0. It showed that retraining could help, but it still did not fully solve the deeper mismatch between old training data and real pipeline conditions.

v3.0.0 Summary

v3.0.0 is the most mature and most production-ready GCN version. It extends the earlier models by supporting multi-AP topologies, multiple segment lengths, and stronger generalization across random seeds. It passed a full 54-experiment evaluation matrix and is the recommended deployment model.

12. Final Conclusion

Across the documented project history, the GCN detector evolved from a promising graph-based classifier into a robust, systematically evaluated production model.

The key progression was:

- from **concept success**
- to **deployment failure**
- to **root-cause discovery**
- to **generalized production readiness**

The final conclusion is:

- **v1.0.0** was the conceptual baseline
- **v2.0.0** exposed critical hidden flaws
- **v2.1.0** was a useful corrective step
- **v3.0.0** is the best version and should be treated as the primary production model

If this needs to be turned into a shorter version for a report, viva, thesis chapter, or presentation slides, it can be condensed into those formats.